ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026

1. Why Normalization Matters

Definition: Organizing a relational schema to reduce redundancy and improve data integrity by decomposing relations based on their functional dependencies.

Each relation should represent **one fact about one entity**.

Three Anomalies (from poor table design):

- **Insertion anomaly**: Cannot add data without unrelated data. *E.g., cannot add a new course unless a section already exists.*
- **Deletion anomaly**: Removing data loses unrelated facts. *E.g., deleting the last section of a course loses the course title.*
- **Update anomaly**: Changing one fact requires updating many rows; partial updates cause inconsistency. *E.g., renaming a department requires changing every row that references it.*

2. Functional Dependencies (FDs)

Definition: A functional dependency $X \to Y$ states: for each value of X, there is **exactly one** value of Y.

X = **determinant** (LHS), Y = **dependent** (RHS).

Formal definition:
$\forall\, t_1, t_2 \in r(R):\ t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$

> FDs come from **business rules**, not from inspecting data instances. A table might happen to satisfy an FD by coincidence; only domain knowledge confirms it.

- **Trivial FD**: $Y \subseteq X$ (always holds, useless for normalization).
- **Non-trivial FD**: $Y \not\subseteq X$ (real constraint from business rules).

3. Armstrong's Axioms

Three core axioms (sound and complete):

- **Reflexivity**: If $Y \subseteq X$, then $X \to Y$.
- **Augmentation**: If $X \to Y$, then $XZ \to YZ$.
- **Transitivity**: If $X \to Y$ and $Y \to Z$, then $X \to Z$.

Two derived rules (provable from the axioms):

- **Union**: If $X \to Y$ and $X \to Z$, then $X \to YZ$.
- **Decomposition**: If $X \to YZ$, then $X \to Y$ and $X \to Z$.

These rules let us derive **all** FDs that logically follow from a given set F. The set of all such derived FDs is called F^+ (the **closure of** F).

4. Attribute Closure (X^+)

Given a set of FDs F and a set of attributes X, the **attribute closure** X^+ is the set of all attributes functionally determined by X.

Algorithm to compute X^+:

1. Initialize: $X^+ = X$.
2. For each FD $Y \to Z$ in F:
   if $Y \subseteq X^+$, then $X^+ = X^+ \cup Z$.
3. Repeat step 2 until no change occurs.

Three key uses of closure:

- **Test if an FD holds**: $X \to Y$ holds under F iff $Y \subseteq X^+$.
- **Test if X is a superkey**: X is a superkey iff $X^+ = R$ (i.e., X determines every attribute in the relation).
- **Find candidate keys**: A **candidate key** is a minimal superkey: no proper subset of it is also a superkey.

Worked Example: $R(A, B, C, D, E)$

$F = \{A \to B,\ BC \to D,\ D \to E\}$

Example 1: Compute $\{A, C\}^+$:

1. $\{A, C\}^+ = \{A, C\}$
2. $A \to B$: $A \in \{A, C\}^+ \to$ add $B \to \{A, B, C\}$
3. $BC \to D$: $\{B, C\} \subseteq \{A, B, C\} \to$ add $D \to \{A, B, C, D\}$
4. $D \to E$: $D \in \{A, B, C, D\} \to$ add $E \to \{A, B, C, D, E\}$
5. No more changes: $\{A, C\}^+ = \{A, B, C, D, E\} = R$

Example 2: Compute $\{B, C\}^+$:

1. $\{B, C\}^+ = \{B, C\}$
2. $A \to B$: $A \notin \{B, C\}^+ \to$ skip
3. $BC \to D$: $\{B, C\} \subseteq \{B, C\} \to$ add $D \to \{B, C, D\}$
4. $D \to E$: $D \in \{B, C, D\} \to$ add $E \to \{B, C, D, E\}$
5. No more changes: $\{B, C\}^+ = \{B, C, D, E\} \neq R$

Example 3: Compute $\{A\}^+$:

1. $\{A\}^+ = \{A\}$
2. $A \to B$: $A \in \{A\} \to$ add $B \to \{A, B\}$
3. $BC \to D$: $C \notin \{A, B\} \to$ skip
4. $D \to E$: $D \notin \{A, B\} \to$ skip
5. No more changes: $\{A\}^+ = \{A, B\} \neq R$

Example 4: Compute $\{D\}^+$:

1. $\{D\}^+ = \{D\}$
2. $A \to B$: $A \notin \{D\} \to$ skip
3. $BC \to D$: $\{B, C\} \not\subseteq \{D\} \to$ skip
4. $D \to E$: $D \in \{D\} \to$ add $E \to \{D, E\}$
5. No more changes: $\{D\}^+ = \{D, E\} \neq R$

5. Finding Superkeys & Candidate Keys

Superkey: A set of attributes X such that $X^+ = R$ (its closure includes every attribute in the relation).

Candidate key (CK): A superkey where no proper subset is also a superkey (i.e., it is **minimal**).

Strategy to find all candidate keys:

1. **Identify "must-have" attributes**: Attributes that **never appear on any RHS** of any FD must be in every CK. Call this set M.
2. **Check if M alone is a superkey**: Compute M^+. If $M^+ = R$, then M is the only CK.
3. **If not, expand M**: For each attribute $A \notin M^+$, try adding it to M and compute the new closure. Continue adding attributes until you get a superkey.
4. **Minimize**: For each superkey found, check that removing any single attribute breaks $X^+ = R$. If so, it is a CK.
5. **Repeat**: Try all combinations to find **every** CK (a relation can have multiple CKs).

Testing minimality:
X is a CK iff $X^+ = R$ **and** $\forall\, A \in X$: $(X - \{A\})^+ \neq R$.

Worked Example: $R(A, B, C, D, E)$

$F = \{A \to B,\ BC \to D,\ D \to E\}$

1. **Identify must-have attributes:**
   RHS attributes across all FDs: $\{B, D, E\}$.
   A and C never appear on any RHS $\to M = \{A, C\}$.
   Both must be in **every** candidate key.
2. **Check if M is a superkey:**
   From **Example 1**: $\{A, C\}^+ = \{A, B, C, D, E\} = R$. ✓
3. **Test minimality:**
   Remove A: $\{C\}^+ = \{C\} \neq R$. ✓
   Remove C: $\{A\}^+ = \{A, B\} \neq R$ (**Example 3**). ✓
   Neither proper subset is a superkey,
   so $\{A, C\}$ is a **candidate key**.
4. **Any other CKs?**
   Since $M = \{A, C\}$ already reaches R and both attributes are mandatory (never on any RHS), no other minimal combination exists.

$\therefore$ The **only candidate key** is $\{A, C\}$.

ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026

6. Canonical Cover (F_c)

The **canonical cover** F_c is a minimal, equivalent set of FDs (same closure as F, but with no redundancy).

Three-step algorithm:

1. **Decompose compound RHS:** Split each FD so every RHS has a single attribute: $X \to YZ$ becomes $X \to Y$ and $X \to Z$.
2. **Reduce extraneous LHS attributes:** For each FD $X \to A$ and each $B \in X$: Compute $(X - \{B\})^+$ under F. If $A \in (X - \{B\})^+$, then B is extraneous. Remove B from the LHS.
3. **Remove redundant FDs:** For each FD $X \to A$ in F: Compute X^+ using $F - \{X \to A\}$. If $A \in X^+$, the FD is derivable from the rest. Remove it.

F_c is the prerequisite input for the 3NF synthesis algorithm.

Worked Example: $R(A, B, C, D)$

$F = \{A \to BC,\ AB \to C,\ A \to B,\ D \to A\}$

1. **Decompose compound RHS:**
   $A \to BC$ becomes $A \to B$ and $A \to C$.
   Result: $\{A \to B,\ A \to C,\ AB \to C,\ A \to B,\ D \to A\}$
   Remove duplicate $A \to B$:
   $F = \{A \to B,\ A \to C,\ AB \to C,\ D \to A\}$
2. **Reduce extraneous LHS attributes:**
   Check $AB \to C$: is B extraneous?
   Compute $(AB - \{B\})^+ = \{A\}^+$:
   $A \to B$ adds B, $A \to C$ adds $C \to \{A\}^+ = \{A, B, C\}$.
   $C \in \{A, B, C\} \to$ yes, B is extraneous.
   $AB \to C$ simplifies to $A \to C$.
   But $A \to C$ already exists $\to$ remove duplicate.
   $F = \{A \to B,\ A \to C,\ D \to A\}$
3. **Remove redundant FDs:**
   Is $A \to B$ redundant? Compute $\{A\}^+$ using $F - \{A \to B\}$:
   Under $\{A \to C,\ D \to A\}$: $\{A\}^+ = \{A, C\}$.
   $B \notin \{A, C\} \to$ not redundant. Keep.
   Is $A \to C$ redundant? Compute $\{A\}^+$ using $F - \{A \to C\}$:
   Under $\{A \to B,\ D \to A\}$: $\{A\}^+ = \{A, B\}$.
   $C \notin \{A, B\} \to$ not redundant. Keep.
   Is $D \to A$ redundant? Compute $\{D\}^+$ using $F - \{D \to A\}$:
   Under $\{A \to B,\ A \to C\}$: $\{D\}^+ = \{D\}$.
   $A \notin \{D\} \to$ not redundant. Keep.

$\therefore F_c = \{A \to B,\ A \to C,\ D \to A\}$

7. First Normal Form (1NF)

Every attribute contains only **atomic** (indivisible) values. No repeating groups, nested tables, or multi-valued attributes.

Steps to test and fix:

1. Inspect each attribute: does any cell hold multiple values (comma-separated lists, arrays, nested structures)?
2. If yes $\to$ **violates 1NF**.
3. **Decompose**:
   - Remove the multi-valued attribute from the original relation.
   - Create a new relation containing: the original PK (as a foreign key) + the multi-valued attribute (now one value per row).
   - The PK of the new relation is the composite of both columns.

Worked Example

- **Before** (violates 1NF):

StudentID	Name	Courses
101	Alice	DB, OS, AI
102	Bob	DB, Networks

  Courses holds comma-separated lists $\to$ not atomic $\to$ violates 1NF.
- **After** (1NF):

StudentID	Name
101	Alice
102	Bob

StudentID	Course
101	DB
101	OS
101	AI
102	DB
102	Networks

  Multi-valued attribute removed from original.
  New relation: PK = (StudentID, Course).

ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026

8. Key Terminology

Prime attribute: An attribute that belongs to at least one candidate key. If an attribute appears anywhere in any CK, it is prime.

Non-prime attribute: An attribute that is not part of any candidate key. These are the attributes most susceptible to redundancy problems.

Full FD: $X \to Y$ where no proper subset of X determines Y. The entire LHS is needed; removing any attribute from X breaks the dependency.

Partial FD: $\exists\, S \subsetneq X$ such that $S \to Y$. Only part of the LHS is enough to determine Y, meaning the dependency does not truly require all attributes in X.

Transitive dependency: $A \to B \to C$ where B is not a superkey and $B \not\to A$. The attribute C depends on the key only indirectly, through the middleman B. This creates update anomalies because changing B forces changes to C.

9. Second Normal Form (2NF)

Definition: 1NF + no non-prime attribute is **partially dependent** on any candidate key.

Only relevant with composite CKs. Single-attribute CKs autopass.

Steps to test and fix:

1. Find all candidate keys using attribute closure.
2. Classify each attribute as prime or non-prime.
3. For each **proper subset** S of each composite CK, compute S^+.
4. If S^+ contains any non-prime attribute $A \to$ partial dependency $\to$ **violates 2NF**.
5. **Decompose (Fix)**: For partial FD $S \to A$, create a new relation $S \cup \{A\}$ with key S. Remove A from the original relation.

Worked Example

$R(A, B, C, D, E)$, $F = \{A \to B,\ BC \to D,\ D \to E\}$

CK = $\{A, C\}$ (from earlier).

Prime: A, C **Non-prime**: B, D, E

1. Proper subsets of CK: $\{A\}$ and $\{C\}$.
2. $\{A\}^+ = \{A, B\}$ (using $A \to B$) $\to$ contains non-prime B.
   Partial dep.: $A \to B \to$ violates 2NF.
3. **Decompose** on $A \to B$:
   - $R_1(\underline{A}, B)$ (key: A)
   - $R_2(\underline{A}, \underline{C}, D, E)$ (key: $\{A, C\}$)
4. Check $\{C\}^+ = \{C\} \to$ no non-prime attrs $\to$ OK.

Result:
$R_1(\underline{A}, B)$ – in 2NF ✓
$R_2(\underline{A}, \underline{C}, D, E)$ – check separately.

10. Third Normal Form (3NF)

Definition: 2NF + for every non-trivial FD $X \to Y$:
X is a superkey **OR** every attribute in Y is prime.

In other words, no non-prime attribute can depend on something that is not a superkey. The only exception is when the dependent attribute is itself part of a candidate key (the "escape clause").

Steps to test and fix:

1. Decompose all FDs to single-attribute RHS.
2. For each FD $X \to A$:
   - Compute X^+. If $X^+ = R$, then X is a superkey, therefore **passes**.
   - Else: is A prime? If yes, then **passes** ("escape clause").
   - Neither, therefore **violates 3NF** (transitive dependency).
3. **Decompose (Fix)**: Create a new relation $X \cup \{A\}$; drop A from the original relation.

Worked Example

$R(G, H, I, J)$, $F = \{G \to HI,\ I \to J\}$

CK = $\{G\}$ (since $\{G\}^+ = \{G, H, I, J\} = R$)

Prime: G **Non-prime**: H, I, J

1. **Decompose to single-attribute RHS:**
   $G \to HI$ becomes $G \to H$ and $G \to I$.
   $F = \{G \to H,\ G \to I,\ I \to J\}$
2. **Test each FD:**
   - $G \to H$: $\{G\}^+ = \{G, H, I, J\} = R$, therefore G is a superkey, therefore **passes**. ✓
   - $G \to I$: $\{G\}^+ = R$ (same closure), therefore **passes**. ✓
   - $I \to J$: $\{I\}^+ = \{I, J\} \neq R$, therefore I is not a superkey.
     Is J prime? No, J is not part of any candidate key.
     Violates 3NF: transitive dependency ($G \to I \to J$). The value of J is determined by I, which is itself determined by the key. This means J is stored redundantly whenever I repeats.
3. **Decompose (Fix)** on $I \to J$:
   - New relation: $I \cup \{J\} = R_1(\underline{I}, J)$ (key: I)
   - Original minus J: $R_2(\underline{G}, H, I)$ (key: G)

   I is the common attribute and is a superkey of R_1, so the join is lossless.

Result:

- $R_1(\underline{I}, J)$ – 3NF ✓
- $R_2(\underline{G}, H, I)$ – 3NF ✓

Every FD now has a superkey on its LHS. No transitive dependencies remain.

11. Boyce-Codd Normal Form (BCNF)

Definition: For every non-trivial FD $X \to Y$, X **must be a superkey**. No exceptions whatsoever: unlike 3NF, there is no prime-attribute escape clause. BCNF is strictly stronger than 3NF. Any relation in BCNF is automatically in 3NF, but not vice versa.

Steps to test and fix:

1. For each non-trivial FD $X \to Y$: compute X^+.
   If $X^+ \neq R$, then X is not a superkey, therefore **violates BCNF**.
2. **Decompose (Fix)**:
   Create two new relations: $X \cup Y$ and $R - (Y - X)$.
3. Recursively test and decompose each new relation until all are in BCNF.

3NF vs. BCNF:

FD $X \to Y$	3NF	BCNF
X is superkey	Pass	Pass
X not SK, Y all prime	Pass	**Fail**
X not SK, Y non-prime	Fail	Fail

The middle row is where 3NF and BCNF diverge. The worked example below targets exactly this case.

Worked Example

$R(A, B, C)$, $F = \{AB \to C,\ C \to B\}$

CKs: $\{A, B\}$ and $\{A, C\}$ (both reach all of R)

Prime: A, B, C – every attribute belongs to at least one CK.

1. Check $AB \to C$:
   $\{A, B\}^+ = \{A, B, C\} = R$, therefore $\{A, B\}$ is a superkey, therefore **passes**. ✓
   This FD has a full candidate key on the LHS, so it causes no issues under any normal form.
2. Check $C \to B$:
   $\{C\}^+ = \{C, B\} \neq R$, therefore C is not a superkey.
   Under 3NF: B is prime, therefore passes via the escape clause.
   Under BCNF: escape clause does not exist, therefore violates BCNF. Even though B is a prime attribute, BCNF requires the determinant to be a superkey, and C alone is not.
3. **Decompose** on the violating FD $C \to B$:
   - $R_1 = X \cup Y = \{C\} \cup \{B\} = (\underline{C}, B)$
     Key: C (the LHS of the violating FD always becomes the key of R_1).
   - $R_2 = R - (Y - X) = \{A, B, C\} - (\{B\} - \{C\}) = \{A, B, C\} - \{B\} = (\underline{A}, \underline{C})$
     Key: $\{A, C\}$ (since $\{A\}^+ = \{A\} \neq R_2$ and $\{C\}^+ = \{C\} \neq R_2$, but $\{A, C\}^+ = R_2$).

   Both R_1 and R_2 are now in BCNF: every non-trivial FD in each has a superkey on the LHS.

Result:

- $R_1(\underline{C}, B)$ – BCNF ✓
- $R_2(\underline{A}, \underline{C})$ – BCNF ✓

ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026

12. 3NF Synthesis (Bottom-Up)

Input: Relation R with FD set F.
Output: A set of relations, all in 3NF.

1. Compute the canonical cover F_c.
2. For each FD $X \to A$ in F_c: create relation $R_i = X \cup \{A\}$. Combine FDs with the same LHS into one relation.
3. If no R_i contains a candidate key of R, add a relation consisting of a candidate key.
4. Remove any R_i that is a subset of another R_j.

Guarantees: Lossless join ✓ Dep. preservation ✓

Worked Example

$R(A, B, C, D, E)$, $F = \{AB \to C,\ C \to B,\ C \to D,\ D \to E\}$

CKs: $\{A, B\}$ and $\{A, C\}$ (verify: $\{A, B\}^+ = \{A, C\}^+ = R$)

1. **Compute F_c:**
   All RHS are already single-attribute.
   Check extraneous LHS in $AB \to C$: is A extraneous? $\{B\}^+ = \{B\}$, $C \notin \{B\}$. No. Is B extraneous? $\{A\}^+ = \{A\}$, $C \notin \{A\}$. No.
   Redundancy check: none are derivable from the rest.
   $F_c = \{AB \to C,\ C \to B,\ C \to D,\ D \to E\}$
2. **Create relations from F_c:**
   $AB \to C$: $R_1(\underline{A}, \underline{B}, C)$
   $C \to B$ and $C \to D$ share the same LHS, combine:
   $R_2(\underline{C}, B, D)$
   $D \to E$: $R_3(\underline{D}, E)$
3. **CK check:**
   Does any R_i contain a CK of R?
   CK $\{A, B\}$: R_1 has A and B. ✓
   No extra relation needed.
4. **Subset check:**
   Is any $R_i \subseteq R_j$?
   $R_1 = \{A, B, C\}$, $R_2 = \{C, B, D\}$, $R_3 = \{D, E\}$.
   No subset relationships. Keep all three.

Result:

- $R_1(\underline{A}, \underline{B}, C)$ – 3NF ✓
- $R_2(\underline{C}, B, D)$ – 3NF ✓
- $R_3(\underline{D}, E)$ – 3NF ✓

Lossless join ✓ Dep. preservation ✓

13. BCNF Decomposition (Top-Down)

Input: Relation R with FD set F.
Output: A set of relations, all in BCNF.

1. If R is already in BCNF, stop.
2. Find a violating FD $X \to Y$ (where X is not a superkey).
3. Decompose:
   $R_1 = X \cup Y, \quad R_2 = R - (Y - X)$.
4. Recursively apply to R_1 and R_2.

Guarantees: Lossless join ✓ Dep. pres. **not guaranteed**

	3NF Synth.	BCNF Decomp.
Lossless join	Always	Always
Dep. preservation	Always	Not always
All FD anomalies gone	Not always	Always

Worked Example

$R(A, B, C, D, E)$, $F_c = \{A \to B,\ A \to C,\ C \to D,\ D \to E\}$

CK = $\{A\}$

1. **Check R for BCNF:**
   $A \to B$: $\{A\}^+ = R$, superkey. ✓
   $A \to C$: $\{A\}^+ = R$, superkey. ✓
   $C \to D$: $\{C\}^+ = \{C, D, E\} \neq R$. Violates BCNF.
2. **Decompose** on $C \to D$:
   - $R_1 = C \cup D = (\underline{C}, D)$
   - $R_2 = R - (D - C) = R - \{D\} = (\underline{A}, B, C, E)$
3. **Check $R_1(\underline{C}, D)$:**
   $C \to D$: $\{C\}^+ = \{C, D\} = R_1$, superkey. ✓
   R_1 is in BCNF.
4. **Check $R_2(\underline{A}, B, C, E)$:**
   FDs on R_2: $\{A \to B,\ A \to C,\ A \to E\}$
   (Since D is gone, $D \to E$ does not apply directly, but $A \to E$ holds because $\{A\}^+ = R$ under the original F, and $E \in R_2$.)
   All FDs have A on LHS, and $\{A\}^+ = \{A, B, C, E\} = R_2$, so A is a superkey. ✓
   R_2 is in BCNF.

Result:

- $R_1(\underline{C}, D)$ – BCNF ✓
- $R_2(\underline{A}, B, C, E)$ – BCNF ✓

Lossless join ✓ But is $D \to E$ preserved? No single relation contains both D and E, so **dep. preservation is lost** for this FD.

14. Lossless Join & Dep. Preservation

Lossless Join Test (non-negotiable):
$R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$
The common attributes must be a superkey of at least one side. Failure produces **spurious tuples** (phantom rows that did not exist in the original data).

Rule: Always decompose on a violating FD. The LHS becomes the common attribute and is a superkey of the new relation.

Dependency Preservation Test:
For each FD $X \to Y$ in F: does some R_i contain **all** attributes of both X and Y? If yes for every FD, then dependencies are preserved.

Worked Example

Verify the 3NF synthesis result (from Box 12):

$R_1(\underline{A}, B, C)$, $R_2(\underline{C}, D)$, $R_3(\underline{D}, E)$
$F_c = \{A \to B,\ A \to C,\ C \to D,\ D \to E\}$

Lossless join:

- $R_1 \cap R_2 = \{C\}$. Is $\{C\}$ a superkey of $R_2(\underline{C}, D)$? Yes ($C \to D$). ✓
- $R_2 \cap R_3 = \{D\}$. Is $\{D\}$ a superkey of $R_3(\underline{D}, E)$? Yes ($D \to E$). ✓

All pairwise joins are lossless. No spurious tuples. ✓

Dependency preservation:

- $A \to B$: $A, B \in R_1$. ✓
- $A \to C$: $A, C \in R_1$. ✓
- $C \to D$: $C, D \in R_2$. ✓
- $D \to E$: $D, E \in R_3$. ✓

Every FD can be checked within a single relation. ✓

Verify the BCNF decomposition result (from Box 13):
$R_1(\underline{C}, D)$, $R_2(\underline{A}, B, C, E)$

Lossless join:

- $R_1 \cap R_2 = \{C\}$. Is $\{C\}$ a superkey of $R_1(\underline{C}, D)$? Yes ($C \to D$). ✓

Lossless. ✓

Dependency preservation:

- $A \to B$: $A, B \in R_2$. ✓
- $A \to C$: $A, C \in R_2$. ✓
- $C \to D$: $C, D \in R_1$. ✓
- $D \to E$: D is in R_1, E is in R_2. No single relation contains both. Not preserved.

To enforce $D \to E$, the application would need a cross-table check or trigger. This is the classic BCNF trade-off.

ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026

15. 3NF Synthesis: Steps 3 & 4 in Action

In the worked example on page 4 (Box 12), Steps 3 and 4 of the 3NF synthesis algorithm did not fire: a candidate key was already present in one of the generated relations, and no relation was a subset of another. This supplementary example is specifically designed so that **both** steps are exercised.

Recall the algorithm:

1. Compute the canonical cover F_c.
2. For each FD $X \to A$ in F_c: create relation $R_i = X \cup \{A\}$. Combine FDs with the same LHS into one relation.
3. If no R_i contains a candidate key of R, **add** a relation consisting of a candidate key.
4. **Remove** any R_i that is a subset of another R_j.

Worked Example: $R(P, Q, S, T)$

$F = \{P \to Q,\ Q \to S,\ S \to Q\}$

Find CKs: $\{P\}^+ = \{P, Q, S\} \neq R$ (cannot reach T).
T never appears on any RHS, so T must be in every CK.
$\{P, T\}^+ = \{P, Q, S, T\} = R$, therefore CK = $\{P, T\}$.

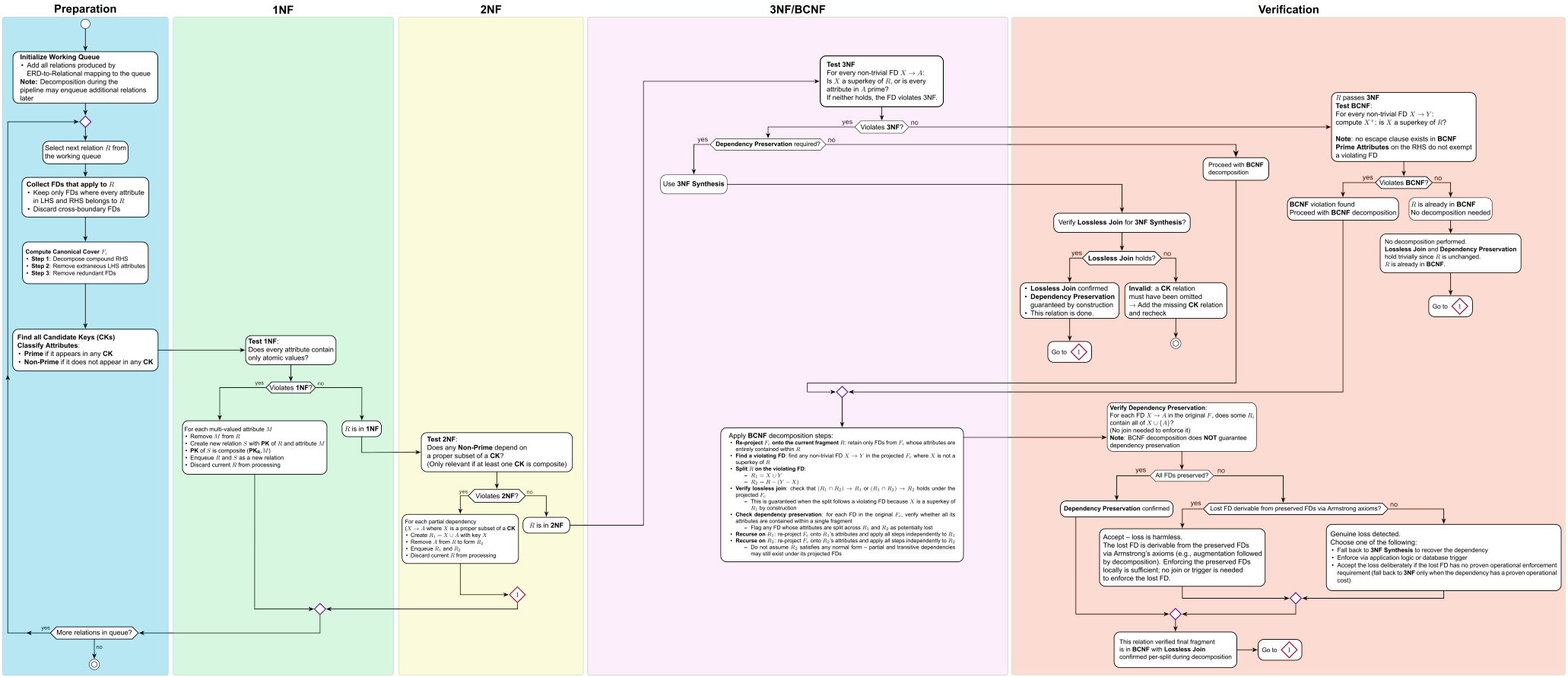
1. **Compute F_c:**
   All RHS are already single-attribute.
   No extraneous LHS attributes (all LHS are single).
   Redundancy check: Is $P \to Q$ redundant? $\{P\}^+$ under $\{Q \to S,\ S \to Q\} = \{P\}$. $Q \notin \{P\}$. Keep. Is $Q \to S$ redundant? $\{Q\}^+$ under $\{P \to Q,\ S \to Q\} = \{Q\}$. $S \notin \{Q\}$. Keep. Is $S \to Q$ redundant? $\{S\}^+$ under $\{P \to Q,\ Q \to S\} = \{S\}$. $Q \notin \{S\}$. Keep.
   $F_c = \{P \to Q,\ Q \to S,\ S \to Q\}$
2. **Create relations from F_c:**
   $P \to Q$: $R_1(\underline{P}, Q)$
   $Q \to S$: $R_2(\underline{Q}, S)$
   $S \to Q$: $R_3(\underline{S}, Q)$
3. **CK check (Step 3 fires)**:
   CK = $\{P, T\}$. Does any R_i contain both P and T?
   $R_1 = \{P, Q\}$, $R_2 = \{Q, S\}$, $R_3 = \{S, Q\}$.
   None contain T, therefore **add** $R_4(\underline{P}, \underline{T})$.
   Without this relation, the decomposition would not be lossless because T would be disconnected from the rest of the schema.
4. **Subset check (Step 4 fires)**:
   $R_3 = \{S, Q\}$ and $R_2 = \{Q, S\}$. Same attributes, therefore $R_3 \subseteq R_2$.
   Remove R_3.
   (The FD $S \to Q$ can still be enforced within R_2 since both S and Q are present.)

Result:

- $R_1(\underline{P}, Q)$ – 3NF ✓
- $R_2(\underline{Q}, S)$ – 3NF ✓
- $R_4(\underline{P}, \underline{T})$ – 3NF ✓ (added by Step 3)

Lossless join ✓ Dep. preservation ✓

Summary: Step 3 added R_4 to include the candidate key $\{P, T\}$, and Step 4 removed R_3 because it was a duplicate of R_2.

ENPM818T – L4-L5: Normalization & Denormalization

Zeid Kootbally | University of Maryland | Spring 2026